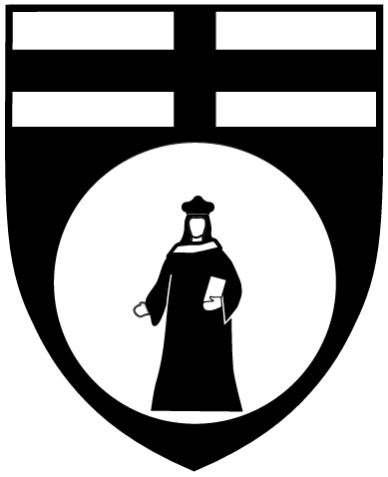

Dibris Dipartimento di Informatica, Bioingegneria,
Robotica e Ingegneria dei Sistemi

Virtual Reality for Robotics - code 104737 - a.y. 2025/2026

Immersive Mapping in Virtual Reality for Robot Teleoperation

Advisor: Subhransu Sourav Priyadarshan

Authors: Rubin Khadka Chhetri s6558048
Sarvenaz Ashoori s6878764
Abdul Hayee Hafiz s6029926

UNIVERSITY OF GENOA, JUNE 2026

Abstract

This project presents a cross-platform Virtual Reality teleoperation system for mobile robots, designed to overcome the spatial awareness limitations of conventional 2D displays through immersive visualization and multi-perspective camera control. The architecture integrates Unity 2022.3 LTS (Windows) for VR rendering and user interaction, Gazebo (Ubuntu 24.04) for physics-based simulation, and ROS2 Jazzy for message passing. Communication between platforms is established via a bi-directional TCP bridge comprising the ROS-TCP-Connector (Unity client) and ROS-TCP-Endpoint (ROS2 server).

To ensure strict environmental correlation and eliminate mapping drift, the system utilizes a pre-processed 3D mesh exported from RTAB-Map as a static environment synchronized across both platforms. The operator uses Meta Quest controllers to transmit velocity commands (`/cmd_vel`) from Unity to Gazebo, while real-time odometry (`/odom`) streams back to maintain sub-frame pose synchronization. Simulated LiDAR data (`/scan`) is concurrently subscribed by Unity to compute real-time proximity distances to surrounding geometry.

Key enhancements include a switchable multi-camera system, a real-time distance-to-wall feedback module, a 2D minimap for spatial orientation, and a dual-indicator ROS connection and data health monitor for operational transparency. Validation confirms tight coordinate alignment and real-time control loop performance, delivering a robust teleoperation framework that prioritizes operator presence, collision avoidance, and deterministic simulation-to-visualization correlation. The system fulfills core immersive mapping objectives while establishing a scalable architecture for future live-SLAM integration.

Contents

1 Introduction

1.1 Problem Statement

Teleoperation of robots requires high levels of situational awareness. The operator must understand the robot's position, orientation, and proximity to obstacles to navigate safely and complete tasks effectively. However, traditional teleoperation interfaces rely on flat 2D displays showing camera feeds from the robot. This approach fundamentally limits the operator's ability to judge distances and avoid collisions. Without natural depth perception, operators frequently misjudge spatial relationships, struggle with obstacle avoidance in cluttered environments, and experience higher cognitive load during navigation tasks.

1.2 Proposed Solution

To address these limitations, this project implements a VR-based teleoperation interface where the operator is immersed in the robot's environment through a Meta Quest headset. Unlike flat screens, VR provides stereoscopic depth perception and head-tracked viewing, allowing the operator to look around and develop an intuitive understanding of spatial relationships. The interface includes multiple camera angles and real-time distance feedback to nearby walls and objects, enabling the operator to monitor wall clearance and prevent collisions during operation.

1.3 Original Pitch vs. Implementation

The original project pitch required a VR interface displaying the robot's *live mapping process* (e.g., occupancy grid, point cloud, semantic map) during teleoperation, with the operator seeing the map being built in real time while controlling the robot.

However, after analyzing the technical constraints, specifically the computational demands of simultaneous live SLAM and VR rendering at 72+ FPS [1], plus cross-platform latency between Windows (Unity) and Ubuntu (ROS2), we made a deliberate design decision. Instead of live mapping, we implemented a static map pre-built with RTAB-Map. The map is built once and exported as a 3D mesh, and loaded into both Gazebo (for simulation) and Unity (for VR visualization). During teleoperation, RTAB-Map does not run; only the static mesh is used.

This architectural adaptation fulfills the core objective of immersive map visualization while guaranteeing deterministic correlation between simulation and visualization layers. To compensate for the absence of live updates, we added switchable multi-camera views (robot POV, left wheel, right wheel, and following camera), real-time distance-to-wall feedback, a 2D minimap for spatial orientation, and a ROS connection and data health monitor for operational transparency. This approach eliminates mapping drift, ensures perfect environmental alignment, and sustains real-time VR performance under practical hardware constraints. While live mapping remains a direction for future work, the static approach proved more robust given the project's time and hardware constraints.

2 System Architecture

2.1 High-Level Architecture & Components

The system employs a distributed architecture spanning two host platforms: a Windows PC executing Unity 2022.3 LTS as the VR frontend, and an Ubuntu 24.04 machine running ROS2 Jazzy [2] with Gazebo as the physics simulation backend. Cross-platform communication is established via the ROS-TCP-Connector [3] (Unity client) and ROS-TCP-Endpoint [4] (ROS2 server), which serialize ROS2 messages over a standard TCP network link.

The operator interacts with the system through a Meta Quest headset tethered to the Windows host. The Meta XR SDK provides the integration layer between Unity and the headset, enabling controller input capture and VR rendering. Unity samples controller inputs, maps them to linear and angular velocity commands, and publishes `/cmd_vel` messages to Gazebo via the TCP bridge. Gazebo applies these velocities to the simulated robot model, computes updated kinematics, and streams odometry data (`/odom`) back to Unity for real-time pose synchronization in the VR environment. Additionally, Gazebo publishes simulated laser scan data from a virtual LiDAR sensor on the `/scan` topic, which Unity subscribes to for real-time distance-to-wall calculations displayed in the VR interface.

To guarantee strict spatial correlation between the simulation and visualization layers, a static 3D mesh exported from RTAB-Map is loaded with identical coordinate origins into both platforms. This decoupled design isolates VR rendering from physics computation, preventing GPU contention and ensuring stable 72+ FPS performance while maintaining deterministic environment alignment.

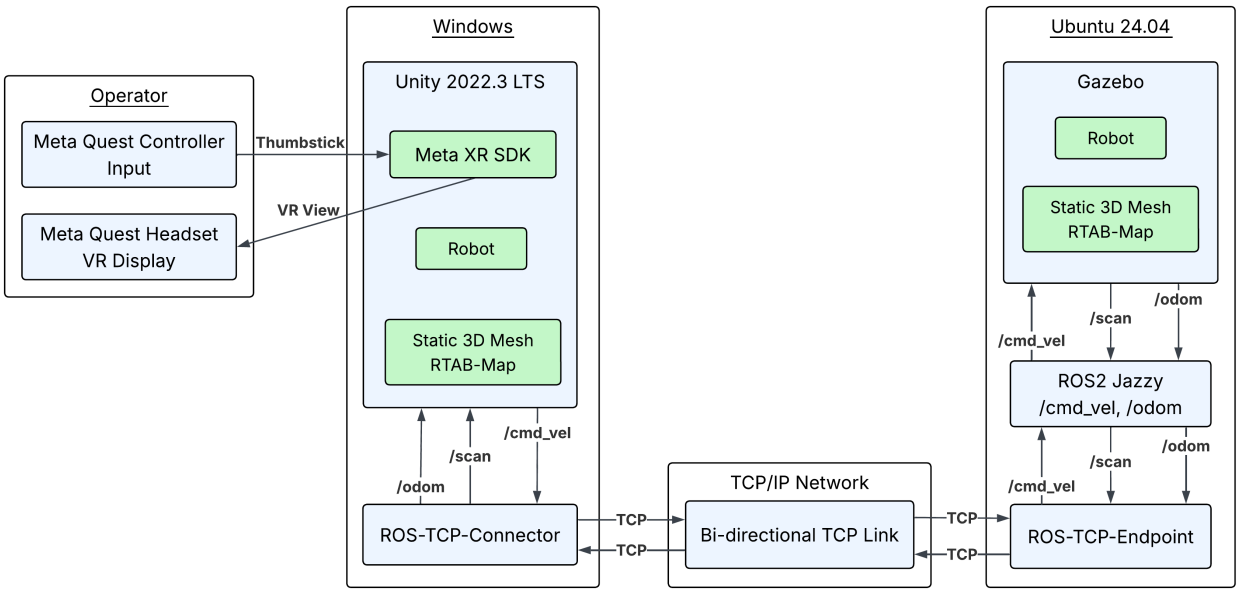

Figure 1: High-level system architecture

Table 1 summarizes the eight core components and their functional responsibilities.

Table 1: System Components and Functional Roles

Component	**Platform**	**Role**
Unity 2022.3 LTS	Windows	VR rendering and interaction layer; handles Meta XR SDK integration, controller input mapping, camera switching and UI overlays
ROS-TCP-Connector	Unity (C#)	Client-side library; serializes ROS2 message structures into TCP byte streams and manages Unity-based publishers/subscribers
ROS-TCP-Endpoint	Ubuntu 24.04	ROS2 bridge node; opens a TCP server port, deserializes incoming byte streams into native ROS2 messages, and publishes to standard topics
ROS2 Jazzy	Ubuntu 24.04	Communication middleware; orchestrates topic routing and message passing between the Endpoint node, Gazebo, and other ROS nodes
Gazebo	Ubuntu 24.04	Physics simulation backend; executes differential drive kinematics, handles collision detection, computes ground-truth odometry (`/odom`), and publishes simulated laser scan data (`/scan`) from a virtual LiDAR
RTAB-Map	Ubuntu (offline)	Pre-processing pipeline; generates a high-fidelity 3D mesh (.obj) from sensor data, exported once as the static environment for both Gazebo and Unity
Meta Quest & Controllers	VR Hardware	OpenXR-compliant I/O; provides head-tracked stereoscopic display and thumbstick-based velocity input for teleoperation
Meta XR Simulator	Windows (Simulator)	OpenXR-compliant I/O; development conducted in simulator with API-identical behavior to physical device

2.2 Data Flow & Communication Pipeline

The teleoperation control loop operates across two isolated hosts, synchronized via the ROS-TCP bridge. Figure 2 illustrates the complete bidirectional sequence from controller input to visual feedback.

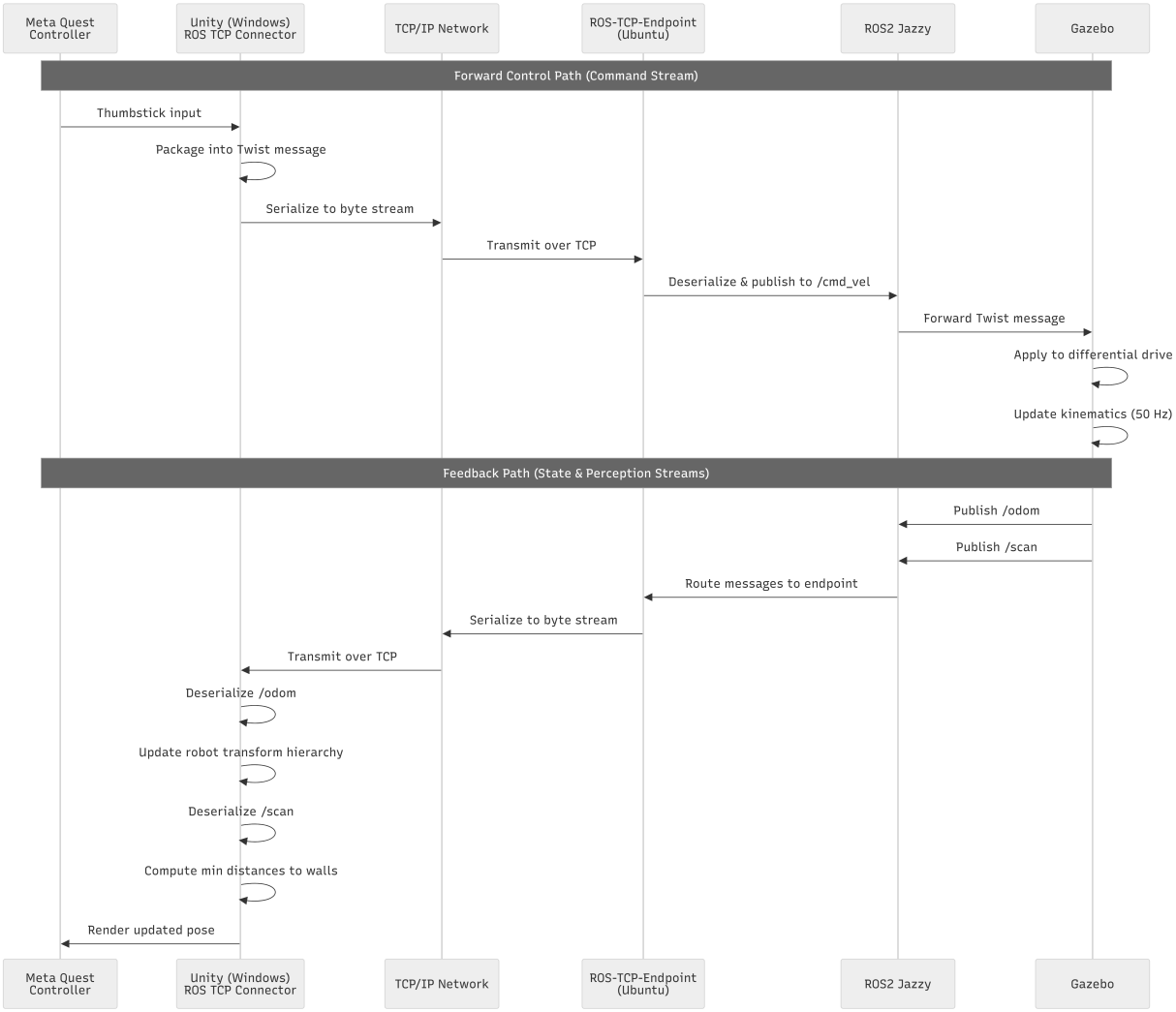

Figure 2: Sequence diagram of the teleoperation control loop

The forward control path begins with Unity sampling Meta Quest thumbstick inputs at the VR frame rate. Analog values are filtered through configurable dead-zones, mapped to linear (v_x) and angular (ω_z) velocities, and packaged into `geometry_msgs/Twist` messages. The ROS-TCP-Connector serializes these commands into TCP byte streams and transmits them to the Ubuntu host. The ROS-TCP-Endpoint node receives the stream, reconstructs the native ROS2 message, and publishes it to the `/cmd_vel` topic. Gazebo's differential drive controller subscribes to this topic, applies the velocity to the robot model, and computes updated kinematics and collision responses against the static environment mesh.

The feedback path operates concurrently. Gazebo publishes ground-truth odometry (`nav_msgs/Odometry`) at 50 Hz and simulated laser scan data (`sensor_msgs/LaserScan`) at 10 Hz back through the ROS-TCP-Endpoint to Unity. Unity consumes the `/odom` stream to update the virtual robot's transform hierarchy, maintaining tight

pose synchronization. Simultaneously, the `/scan` subscriber processes incoming range data to compute minimum distances to surrounding geometry. The 10 Hz scan rate provides sufficient temporal resolution for collision-aware navigation at typical teleoperation speeds while minimizing network bandwidth and CPU overhead.

3 Mapping & Environment Preparation

3.1 Mapping Pipeline & Process

The environment map is generated offline using RTAB-Map prior to teleoperation. Live SLAM was avoided because continuous point cloud updates would lag behind the robot's position. Even with Gazebo running on a separate machine, Unity must maintain 72 Hz VR rendering, and live map updates would always trail the robot's actual movement. By the time a new region is visualized, the robot may have already moved past it, degrading operator situational awareness. The static map eliminates this update latency while reserving all computational resources for teleoperation performance.

The TurtleBot3 Waffle model is used for mapping due to its reliable wheel odometry in simulation. Its URDF was modified to include an RGB-D camera sensor for RTAB-Map input.

The mapping procedure follows these steps:

1. **Data Collection:** The TurtleBot3 Waffle is manually driven through the environment using keyboard teleoperation. RTAB-Map subscribes to synchronized RGB-D, LiDAR, and odometry streams using the launch configuration described in Section 3.2.

2. **Point Cloud Generation:** RTAB-Map performs graph-based SLAM with loop closure detection, building a globally consistent 3D point cloud from aligned sensor frames.

3. **Mesh Reconstruction:** The RTAB-Map Database Viewer GUI (Figure 3) processes the point cloud (Figure 4) via surface reconstruction to generate a watertight, triangulated 3D mesh [5] (Figure 5). Unlike raw point clouds, which lack continuous surface topology and cannot participate in physics calculations, the triangulated mesh provides the closed geometry required for accurate collision response in Gazebo.

4. **2D Map Export:** A top-down 2D occupancy grid is exported from the same session and assigned as the texture for the Unity minimap overlay (Figure 11).

The identical `.obj` mesh is loaded into both Gazebo (for collision) and Unity (for visual alignment).

All modified and newly created files, including the updated TurtleBot3 URDF with RGB-D camera, the RTAB-Map launch configuration, and the exported static mesh are available in the project GitHub repository [6].

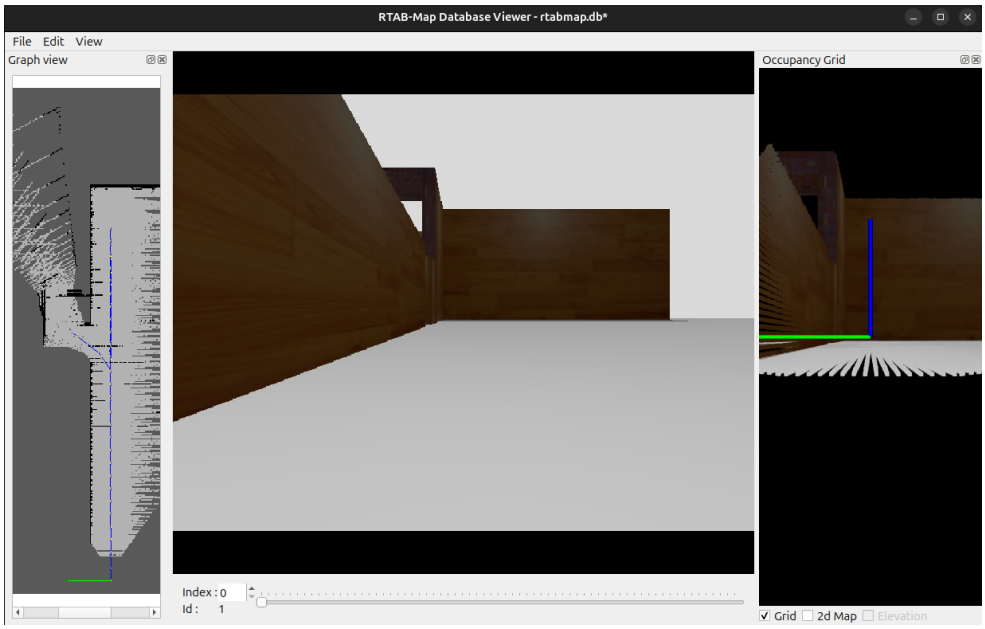

Figure 3: RTAB-Map Database Viewer GUI used for mesh reconstruction and 2D map export

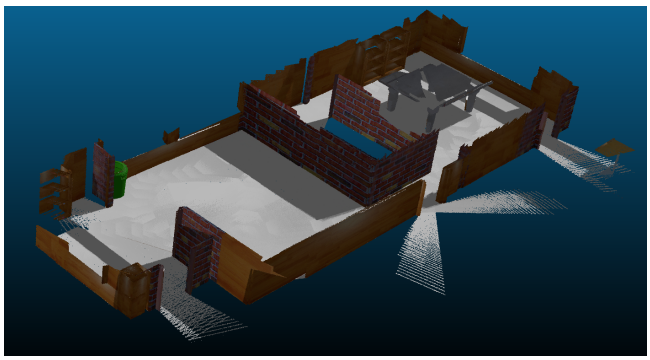

Figure 4: Static 3D point cloud from RTAB-Map

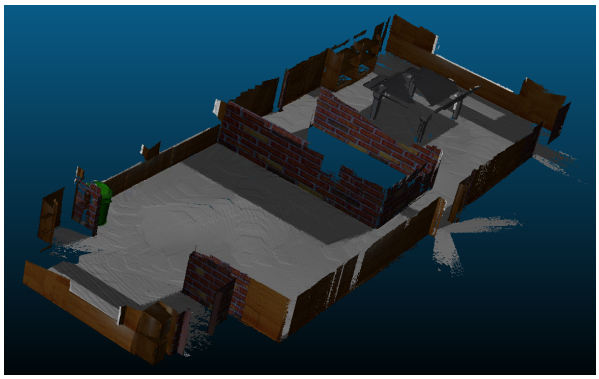

Figure 5: Final 3D mesh exported from point cloud

3.2 Challenges in Map Generation

Generating a globally consistent static map proved to be the most labor-intensive phase of the project, requiring iterative mapping sessions and extensive parameter tuning. A video recording of a mapping attempt is available in the project repository and at:

https://github.com/user-attachments/assets/681ad4d4-4473-4bb8-8107-3008278fa6f9

Two primary challenges were encountered.

Odometry Drift: Despite using the TurtleBot3 Waffle for its reliable simulated wheel odometry, cumulative drift accumulated over extended trajectories, causing misalignment in the point cloud and distorted mesh

outputs. To mitigate this, mapping was restricted to controlled loops. The robot was driven at reduced speeds (< 0.2 m/s) with frequent pauses to allow RTAB-Map's graph optimizer to converge on loop closures before proceeding.

Parameter Tuning: Default RTAB-Map configurations produced fragmented meshes and spatially inconsistent point clouds. After extensive testing, a final configuration was established that balanced feature extraction, loop closure sensitivity, and voxel resolution. Key adjustments included:

- Disabling visual and ICP odometry (`visual_odometry:=false`, `icp_odometry:=false`) to rely exclusively on ground-truth Gazebo odometry and prevent conflicting pose estimates.
- Reducing the detection rate to 2 Hz (`Rtabmap/DetectionRate:=2`) to maintain stable registration without saturating CPU resources.
- Setting a conservative loop closure threshold (`RGBD/LoopThr:=1.0`) to suppress false positives in feature-poor corridors.
- Configuring a fine voxel resolution (`mesh_output_voxel_size:=0.03`) to preserve wall geometry detail for accurate collision response.

This systematic tuning ensured the exported mesh provided reliable collision geometry for Gazebo and a stable spatial reference for the VR environment.

3.3 Map Deployment

The exported assets are deployed to their respective platforms as follows:

- **3D Mesh to Gazebo:** The `.obj` file is loaded as a static collision model via the `<mesh>` tag in the world SDF file. Gazebo's physics engine uses the triangulated surface to compute accurate wall collision response.
- **3D Mesh to Unity:** Unity natively supports the `.obj` format, allowing the mesh to be imported directly into the scene hierarchy. This ensures immediate visual parity with the simulation environment without intermediate conversion steps.
- **2D Occupancy Grid to Unity:** The exported `.pgm` occupancy grid is converted to `.png` format for native Unity `RawImage` compatibility. The origin and resolution metadata from the `.yaml` file are preserved to guarantee global alignment with the 3D environment.

To verify that the static map is not tied to a specific robot model, the system was tested with a secondary robot after generating the map using the TurtleBot3 Waffle. Figure 6 shows this alternative robot spawned in Gazebo on the identical static mesh, confirming that the environment and collision pipeline are robot-independent. In Unity, the new robot was imported using the URDF-Importer package from a `.urdf` file, enabling consistent cross-platform robot representation without manual mesh reconstruction.

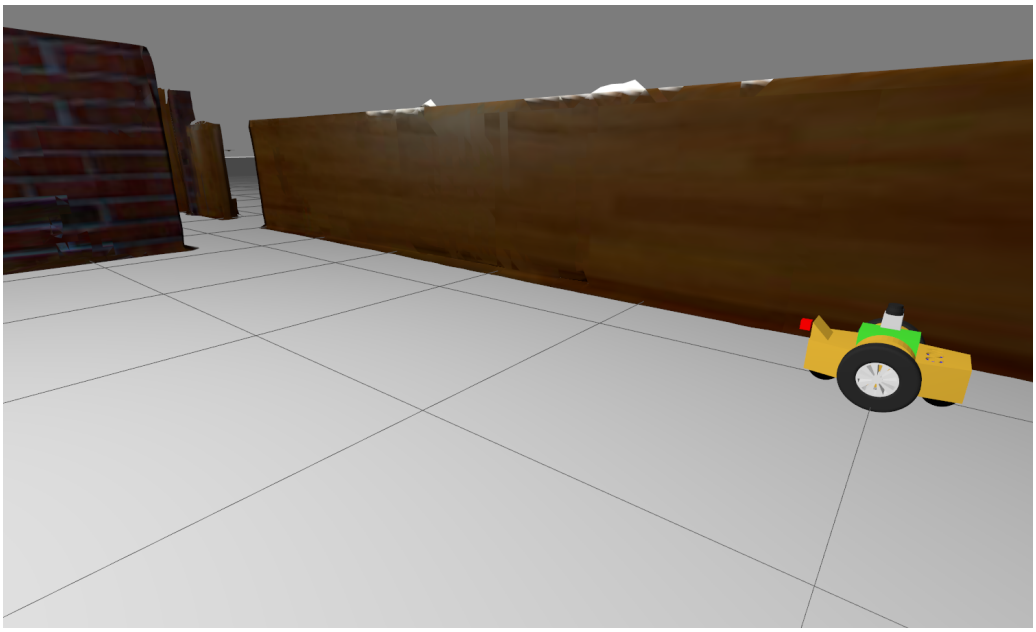

Figure 6: Alternative robot model spawned in Gazebo on the static 3D mesh

4 VR Interface Design

Due to hardware availability constraints, the entire VR interface was developed and tested using the Meta XR Simulator. The simulator uses the same Meta XR SDK and OpenXR API as physical Quest devices, ensuring that all implemented features are directly deployable to a physical headset without code modifications. The following components describe the interface as experienced in the simulator viewport.

4.1 Multi-Camera System

The VR interface provides four switchable camera views to enhance situational awareness and assist the operator in collision avoidance: Main View, Left Wheel View, Right Wheel View, and Following View.

The Main View (Figure 7) attaches the camera to the robot's `camera_link` frame at zero offset, providing an immersive first-person perspective aligned with the robot's coordinate system. The Left Wheel View (Figure 8) parents the camera to the left wheel transform with a calibrated positional offset, allowing the operator to monitor wall clearance on the left side during narrow corridor traversal. Similarly, the Right Wheel View (Figure 9) parents the camera to the right wheel transform for monitoring right-side wall proximity. The Following View (Figure 10) attaches the camera to the LiDAR frame (`scan_link`) with an offset that provides a third-person perspective trailing the robot, offering global context for path planning.

The offset values are calibrated rather than hardcoded to absolute coordinates, providing flexibility to adapt the camera system to different robot models without code modifications.

VR platforms permit only one active camera rig per scene. To enable multiple viewing perspectives without

instantiating additional cameras, the system dynamically reparents the single `OVRCameraRig` to different transform targets. When a view is selected, the camera rig's parent is updated to the corresponding transform (e.g., `left_wheel`, `right_wheel`, or `scan_link`), a local position offset is applied, and the local rotation is reset to maintain a stable orientation relative to the target. This reparenting approach avoids the performance overhead of multiple active cameras while providing seamless view switching.

Figure 7: Main view (robot first-person perspective)

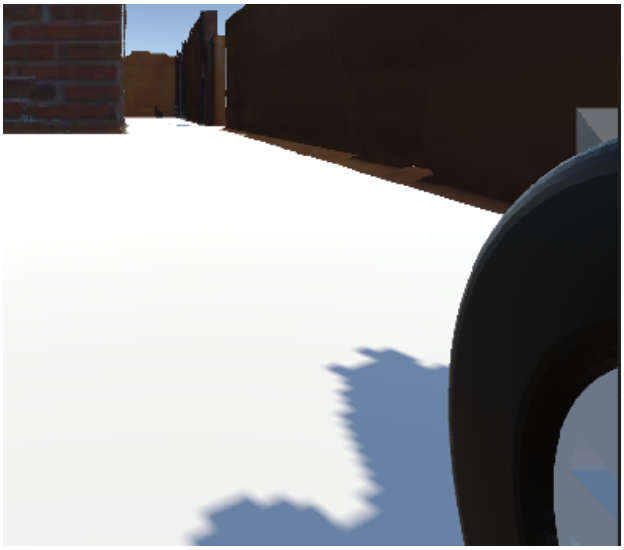

Figure 8: Left wheel view monitoring wall clearance

Figure 9: Right wheel view monitoring wall clearance

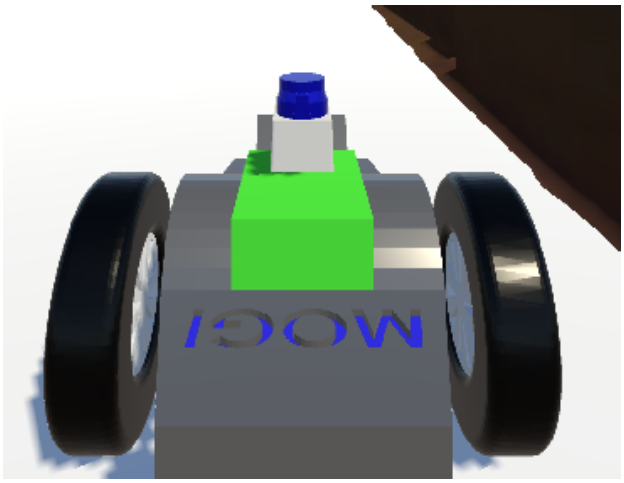

Figure 10: Following view (third-person perspective)

The operator cycles through the four views using dual input methods: pressing the `C` key or performing a left mouse click in the simulator viewport. This dual-input design ensures accessibility regardless of the available input device during development and testing.

This multi-perspective design directly compensates for the absence of live map updates by enabling direct visual inspection of critical clearance zones around the robot, particularly when navigating tight corridors or performing close-quarters maneuvers.

4.2 Minimap

A 2D minimap overlay provides the operator with continuous spatial orientation using the occupancy grid exported by RTAB-Map (as described in Section 3.3). The minimap is anchored to a fixed corner of the VR viewport, remaining visible during all camera views without obstructing the 3D environment.

From the accompanying `.yaml` file, we extract the map metadata: the origin coordinates $(\text{origin}_x, \text{origin}_y)$ defining the bottom-left corner of the map in world coordinates, and the `resolution` (meters per pixel). These values are essential for projecting the robot's real-time position onto the minimap texture.

The robot's current position is obtained from the `/odom` topic. For each received `OdometryMsg`, the robot's world-frame coordinates (x, y) and yaw angle (θ) are extracted. The corresponding pixel coordinates on the minimap texture are calculated as:

$$\text{pixel}_x = \frac{x - \text{origin}_x}{\text{resolution}}, \quad \text{pixel}_y = \frac{y - \text{origin}_y}{\text{resolution}} \tag{1}$$

These pixel coordinates are then converted to UI canvas coordinates. Given the minimap texture dimensions $(\text{width}, \text{height})$ and the canvas size C (set to 700×700 pixels), the conversion is:

$$\text{ui}_x = \left(\frac{\text{pixel}_x}{\text{width}} - 0.5\right) \times C, \quad \text{ui}_y = \left(\frac{\text{pixel}_y}{\text{height}} - 0.5\right) \times C \tag{2}$$

The resulting position is clamped to the canvas bounds $[-C/2, C/2]$ to keep the robot marker visible even when the robot approaches the map edges. The canvas size of 700×700 pixels was chosen as it provides sufficient resolution for the minimap texture while remaining compact enough not to obstruct the operator's 3D view.

The robot is represented on the minimap by a red circular marker (a Unity `Image` component) whose anchored position is updated using the computed $(\text{ui}_x, \text{ui}_y)$ coordinates. To maintain intuitive orientation, the marker is rotated by $-\theta$ (negative robot yaw), ensuring the red dot always points in the robot's forward direction on the top-down map.

Figure 11 shows the minimap as rendered in the VR interface. The red dot indicates the robot's current position and heading, while the grayscale background represents the static environment map. This top-down reference complements the multi-camera system by providing persistent global context, enabling the operator to plan paths and maintain awareness of the robot's location relative to the full workspace.

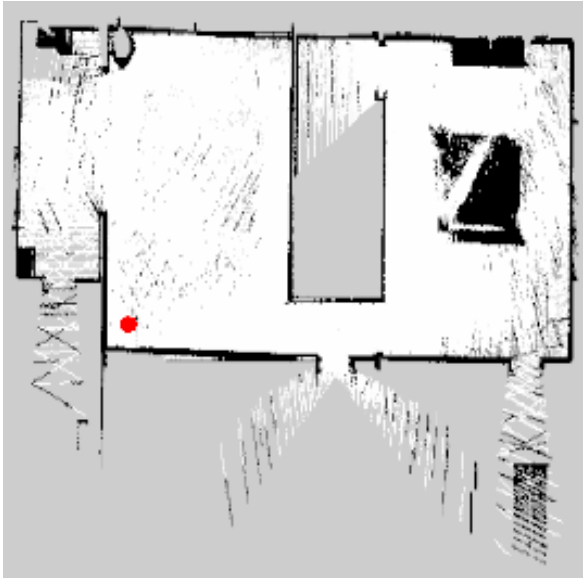

Figure 11: Minimap overlay showing robot position (red dot)

4.3 Distance-to-Wall Feedback

Real-time proximity feedback is provided to the operator using data from the robot's virtual LiDAR sensor. Unity subscribes to the `/scan` topic published by Gazebo and processes incoming `LaserScanMsg` messages to compute minimum distances to walls in four key directions: forward, left, right, and back.

The `LidarScanSubscriber` script establishes a subscription to the `/scan` topic via ROS TCP Connector. Each received `LaserScanMsg` contains an array of range measurements distributed across the sensor's field of view, along with metadata including `angle_min`, `angle_max`, and `angle_increment`.

For each of the four cardinal directions: forward (0°), left (90°), right (-90°), and back (180°), the corresponding range value is extracted by calculating the array index from the target angle:

$$\text{index} = \text{round}\left(\frac{\theta_{\text{target}} - \text{angle}_{\text{min}}}{\text{angle}_{\text{increment}}}\right) \tag{3}$$

The index is clamped to valid array bounds. Invalid measurements (infinity or NaN) are replaced with a configurable maximum distance. In our implementation, this is set to 5.0 meters, which is adjustable based on the environment.

The extracted distances are displayed as UI text elements in the VR overlay. Each text element shows both the direction label (FORWARD, LEFT, RIGHT, BACK) and the measured distance. Color coding provides intuitive risk assessment (Figure 12):

- **Green:** Distance $\geq$ 2.0 meters (safe)
- **Orange:** 1.0 m $\leq$ distance $<$ 2.0 m (warning)

- **Yellow:** 0.5 m $\leq$ distance $<$ 1.0 m (danger)
- **Red:** Distance $<$ 0.5 m (critical)

This color-coded approach reduces the operator's cognitive load during teleoperation. Rather than focusing on precise numerical distances, the operator can instantly perceive the risk level from the displayed color. This allows the operator to maintain attention on the robot's trajectory while still receiving effective collision warnings.

Figure 12: Distance-to-wall feedback UI showing real-time proximity in four directions

4.4 ROS Connection & Data Health Monitor

To provide operational transparency, the VR interface includes a connection state monitor that displays the current status of the ROS-TCP bridge. The `ROSConnectionMonitor` script tracks two independent signals: TCP socket health and message freshness.

Connection Status (TCP Layer): The script periodically checks the `HasConnectionError` property of the `ROSConnection` instance. This flag indicates whether the TCP connection to the ROS-TCP-Endpoint is alive. Based on this, the UI displays:

- **Connecting (Figure 13):** Initial state while awaiting connection establishment (displayed in blue)
- **Connected (Figure 14):** TCP connection successfully established (displayed in green)
- **Disconnected (Figure 16):** No active TCP connection (displayed in red)

Message Freshness (Application Layer): Once connected, the monitor tracks the time elapsed since the last ROS message was received. The `RecordMessageReceived()` method is called by the `LidarScanSubscriber` each time a `/scan` message arrives, updating a timestamp. Scan data was chosen over odometry for freshness

monitoring because the LiDAR topic publishes at a steady rate regardless of robot movement, providing a consistent heartbeat signal. Based on the time since the last message, the UI displays:

- **Active (Figure 14):** `/scan` message received within the last 1 second (green)
- **Delayed:** No message received for 1–3 seconds, indicating possible network or simulation lag (blue)
- **Stale (Figure 15):** No message received for more than 3 seconds, indicating the topic may have stopped publishing (red)

Figure 13: Connecting

Figure 14: Connected

Figure 15: Stale

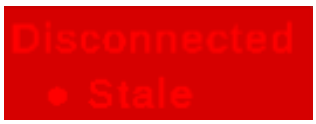

Figure 16: Disconnected

The dual-indicator approach enables precise fault diagnosis. For example, a *Connected* status with *Stale* LiDAR freshness indicates that the network link is intact but the simulated LiDAR sensor or `/scan` publisher has stalled, a scenario invisible to a single "online/offline" indicator. This visibility directly supports safe teleoperation by preventing the operator from relying on outdated or missing proximity data.

4.5 Robot Movement & Pose Synchronization

A fundamental architectural principle is that the robot's motion in Unity is *not* driven directly by VR controller input. Instead, Unity acts as a visualization client that follows the ground-truth pose published by Gazebo. This separation ensures that physics simulation remains the sole authority of Gazebo, while Unity provides a synchronized, immersive representation of the robot's state.

Unity subscribes to the `/odom` topic via the ROS-TCP-Connector. Upon receiving an `OdometryMsg`, the system extracts the robot's world-frame position and orientation. These values are converted from the ROS coordinate convention (X-forward, Y-left, Z-up) to Unity's convention (X-right, Y-up, Z-forward) using the `From<FLU>()` extension method provided by the ROSGeometry library.

To establish a stable reference frame, the first received odometry message is stored as the initial pose $(\mathbf{p}_0, \mathbf{q}_0)$. All subsequent poses are expressed as relative transformations:

$$\mathbf{p}_{\text{rel}} = \mathbf{p}_{\text{current}} - \mathbf{p}_0, \quad \mathbf{q}_{\text{rel}} = \mathbf{q}_{\text{current}} \cdot \mathbf{q}_0^{-1} \tag{4}$$

This relative formulation ensures that the Unity robot moves consistently with the simulated robot regardless of absolute world origin differences between the two platforms.

Several filtering strategies are applied to enhance visual stability in the VR view:

- **Axis Locking:** The vertical position (Y-axis) and roll/pitch rotations (X and Z axes) can be optionally locked to fixed values. This prevents visual artifacts such as robot "bobbing" or tilting that may arise from

minor physics instabilities or mesh imperfections, while preserving the critical horizontal-plane motion and yaw orientation needed for teleoperation.

- **Threshold Filtering:** Pose updates are only applied when the change exceeds a configurable threshold (0.001 m for position, 0.1° for rotation). This suppresses high-frequency noise from the simulator without introducing perceptible latency.
- **Interpolation Smoothing:** The target pose is interpolated using linear and spherical interpolation (`Vector3.Lerp` and `Quaternion.Slerp`) with a smoothing factor of 0.3. This produces visually smooth motion that feels natural in VR while maintaining tight synchronization with the simulation.

The final pose is applied to the robot GameObject using `ArticulationBody.TeleportRoot()` when available (for physics-aware movement), or direct transform manipulation as a fallback. This ensures that the visual robot remains perfectly aligned with the simulated robot's ground-truth state.

This subscribe-to-odometry pattern guarantees that the VR visualization remains a faithful, deterministic representation of the physics simulation.

4.6 Controller Mapping

The operator controls the robot using the Meta Quest controller thumbstick. The thumbstick deflection is mapped to linear and angular velocity commands.

Primary Input (VR Controller):

Unity reads the thumbstick axes using `OVRInput.Get(OVRInput.Axis2D.PrimaryThumbstick)`. The vertical axis (Y) controls linear velocity (forward/backward), while the horizontal axis (X) controls angular velocity (left/right turn). The raw input values are scaled by configurable maximum speeds:

$$v_{\text{linear}} = \text{thumbstick}_y \times \text{maxLinearSpeed}, \quad \omega_{\text{angular}} = \text{thumbstick}_x \times \text{maxAngularSpeed} \tag{5}$$

In the current configuration, `maxLinearSpeed` is set to 0.2 m/s and `maxAngularSpeed` to 0.2 rad/s. These values can be adjusted based on operator preference or environment constraints.

Keyboard Fallback:

To support development and testing without a physical VR headset, a keyboard fallback is implemented. When `useKeyboardFallback` is enabled and no controller input is detected, the script accepts standard WASD keyboard input: **W** moves forward, **S** moves backward, **A** turns left, and **D** turns right.

This dual-input design ensures that teleoperation development and debugging can continue even when the VR headset is not available, while the primary controller input remains functional for actual VR operation.

The resulting linear and angular velocities are packaged into a `TwistMsg` and published to the `/cmd_vel` topic via ROS TCP Connector at the Unity frame rate (approximately 72 Hz). These commands are sent to Gazebo, which applies them to the robot's differential drive controller, completing the teleoperation loop.

5 Results & Validation

The system was validated through functional testing in the Meta XR Simulator with Gazebo running on Ubuntu 24.04. Validation focused on verifying that all subsystems operate correctly together and that the end-to-end teleoperation loop functions as designed.

5.1 Control Loop & Message Flow Validation

The bidirectional teleoperation loop was verified by monitoring topic activity using `ros2 topic hz` and observing system response.

Command Path

Keyboard input (WASD) in Unity generated `geometry_msgs/Twist` messages published to `/cmd_vel`. Figure 17 shows the measured publication rate from Unity. When driving at full speed, the command topic published at approximately 71-72 Hz, varying slightly with the Unity frame rate. Gazebo's differential drive controller received these commands and executed physically consistent motion against the static mesh.

```
shahin@shahin-ASUS-Zenbook-14-UX3405CA-UX3405CA:~$ ros2 topic hz /cmd_vel
average rate: 72.017
        min: 0.000s max: 0.083s std dev: 0.01599s window: 73
average rate: 71.938
        min: 0.000s max: 0.083s std dev: 0.01543s window: 145
average rate: 72.048
        min: 0.000s max: 0.083s std dev: 0.01550s window: 218
average rate: 71.647
        min: 0.000s max: 0.083s std dev: 0.01537s window: 289
average rate: 71.463
```

Figure 17: `ros2 topic hz /cmd_vel` output showing publication rate from Unity (71-72 Hz at full speed)

Feedback Path

Gazebo published odometry and laser scan data at steady rates. Figures 18 and 19 show the measured publication rates.

```
shahin@shahin-ASUS-Zenbook-14-UX3405CA-UX3405CA:~$ ros2 topic hz /odom
average rate: 48.302
        min: 0.001s max: 0.038s std dev: 0.01217s window: 49
average rate: 47.013
        min: 0.001s max: 0.046s std dev: 0.01341s window: 96
average rate: 46.664
        min: 0.000s max: 0.046s std dev: 0.01365s window: 142
average rate: 47.297
        min: 0.000s max: 0.046s std dev: 0.01372s window: 193
average rate: 47.203
```

Figure 18: `ros2 topic hz /odom` output showing odometry publication rate (47-48 Hz)

```
shahin@shahin-ASUS-Zenbook-14-UX3405CA-UX3405CA:~$ ros2 topic hz /scan
average rate: 9.159
        min: 0.096s max: 0.142s std dev: 0.01228s window: 11
average rate: 9.086
        min: 0.096s max: 0.142s std dev: 0.00972s window: 20
average rate: 9.326
        min: 0.094s max: 0.142s std dev: 0.00944s window: 30
average rate: 9.343
        min: 0.093s max: 0.142s std dev: 0.01012s window: 40
average rate: 9.277
```

Figure 19: `ros2 topic hz /scan` output showing LiDAR scan publication rate (9-10 Hz)

Odometry was published at approximately 47-48 Hz, while laser scan data was published at 9-10 Hz. The system is configured to publish odometry at a maximum of 50 Hz and laser scan data at 10 Hz, so the measured rates are within acceptable ranges. The slight reduction in odometry rate is likely due to the combined load of ROS2 message processing and the TCP bridge on the Ubuntu machine. Unity successfully subscribed to both topics via the ROS-TCP-Connector. No message backlog, duplication, or deserialization errors were observed during testing.

Pose Synchronization

The virtual robot in Unity followed the simulated robot in Gazebo with no perceptible lag or spatial drift. Axis locking and interpolation smoothing eliminated visual jitter while preserving responsive control.

Side-by-side observation, shown in Figure 20, confirmed identical position and orientation across platforms, validating that the coordinate transformation pipeline and relative-pose formulation maintain alignment.

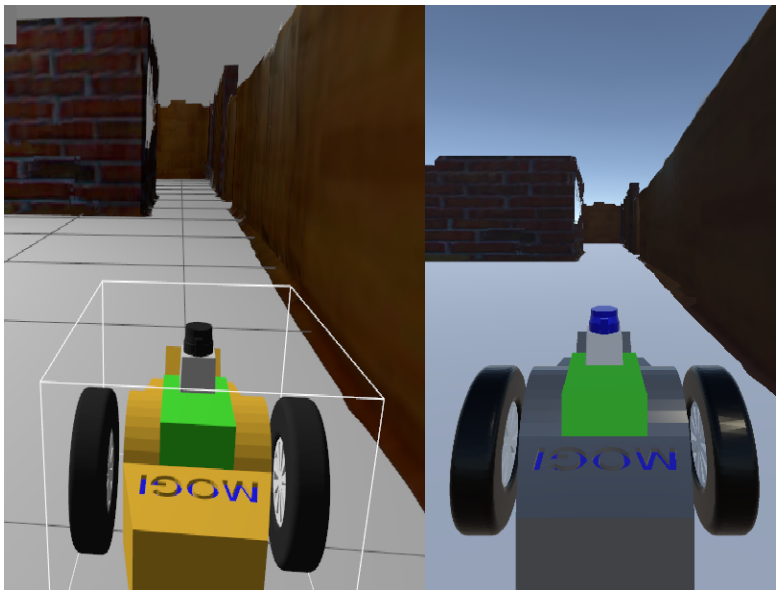

Figure 20: Side-by-side comparison showing the robot's position and orientation in Gazebo (left) and Unity (right) are perfectly synchronized

5.2 Subsystem Functional Verification

Each interface component was validated through targeted observation during teleoperation sessions:

- **Multi-Camera Switching:** Pressing `C` or performing a left mouse click cycled instantly between the Main, Left Wheel, Right Wheel, and Following views. Each perspective rendered correctly with no visual artifacts. The transform reparenting approach completed within a single frame, confirming zero-latency switching without disrupting VR rendering. Figures 7–10 demonstrate all four operational views captured during testing.
- **Minimap:** The red robot marker accurately tracked position and heading across the full workspace. Coordinate conversion from ROS world coordinates to UI canvas coordinates remained stable during extended navigation. The $-\theta$ marker rotation consistently maintained intuitive forward alignment on the top-down map. Figure 11 shows the minimap overlay with the robot marker correctly positioned and oriented.
- **Distance-to-Wall Feedback:** Color-coded proximity values updated at the LiDAR publication rate (9–10 Hz) and correctly reflected distances to walls in all four cardinal directions. Invalid range values (∞ or NaN) were properly filtered and replaced with the configured maximum distance (5.0 m). Threshold-based color transitions (green $\rightarrow$ orange $\rightarrow$ yellow $\rightarrow$ red) triggered at the expected distances during controlled approach maneuvers. Figure 12 shows the feedback overlay displaying real-time proximity values with appropriate color coding.
- **Connection Monitor:** The dual-indicator system correctly displayed TCP connection state and `/scan` message freshness. Simulated network interruptions (stopping the ROS-TCP-Endpoint) triggered appropriate transitions (Connected, Disconnected, Active, Stale), validating the fault-diagnosis capability and preventing operator reliance on stale sensor data. Figures 13–16 demonstrate all four monitor states (Connecting, Connected, Stale, Disconnected) as observed during testing.

5.3 System Demonstration

A complete teleoperation session was recorded from the Meta XR Simulator viewport. Figures 21 and 22 show two different camera perspectives during teleoperation: the first-person robot view and the following third-person view.

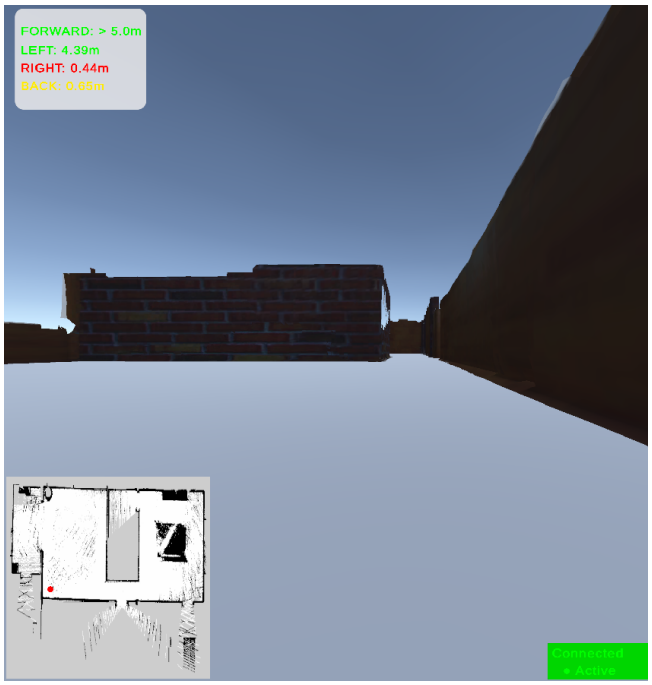

Figure 21: First-person robot view during teleoperation, showing the robot's perspective

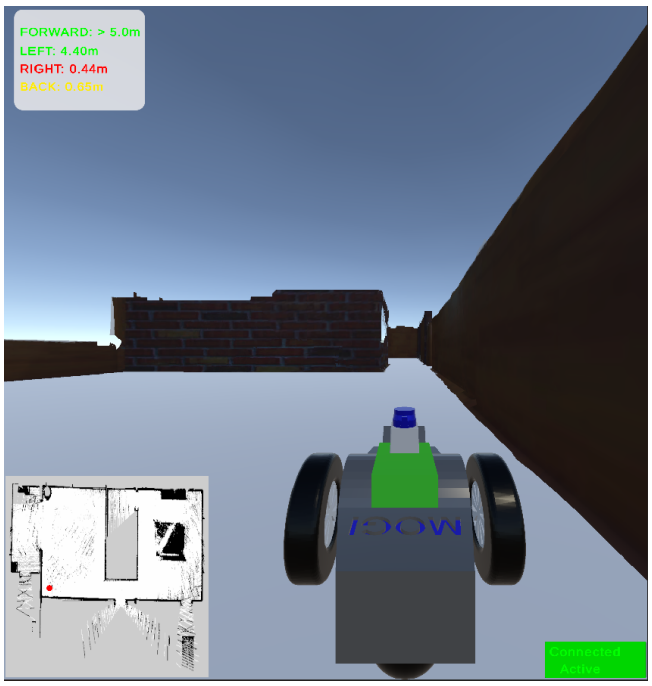

Figure 22: Following view showing the robot from a third-person perspective, with minimap and distance feedback visible

The demonstration video is available in the project repository [7] and at:
https://github.com/user-attachments/assets/e6e9bcdb-ee10-4410-992b-40227894b76e
The video and figures together demonstrate:

- Robot navigating through the static 3D environment using WASD keyboard controls
- Multiple camera perspectives (first-person and following views)
- The minimap tracking robot position and heading in real time
- Distance feedback updating and changing colors as the robot approaches walls
- The connection monitor displaying "Connected" and "Active" status during normal operation
- Stable VR visualization without crashes

Collectively, the functional validation of message flow, pose synchronization, interface components, and end-to-end teleoperation confirms that the system successfully delivers immersive robotic control with deterministic environmental correlation, real-time proximity awareness, and operational transparency. This establishes a robust and reproducible foundation for future extensions, including live mapping and physical hardware deployment.

6 Challenges & Solutions

Throughout the development of this teleoperation system, several technical challenges were encountered. This section details the most significant problems and the solutions implemented to overcome them.

6.1 Cross-Platform TCP Bridge Reliability

Challenge: Initially WiFi was used to establish communication between Windows (Unity) and Ubuntu (ROS2). However, WiFi proved extremely unreliable for real-time teleoperation: weak signal strength caused intermittent connection drops, packet loss, and unpredictable latency. These issues manifested as delayed robot responses, jittery visualization, and occasional complete loss of control.

Solution: To eliminate wireless instability, the two machines were directly connected via a shielded Ethernet cable. This provided a dedicated, low-latency, lossless link between the Windows and Ubuntu hosts. Static IP addresses were configured on both machines (e.g., 192.168.1.x subnet) to prevent IP renewal conflicts and ensure deterministic name resolution.

The ROS-TCP-Endpoint was configured with a fixed port (`tcp_port:=10000`) and explicit client IP whitelisting. Unity implements automatic reconnection logic with 1 Hz retry intervals. Quality of Service (QoS) policies were set to `VOLATILE` with history depth 1 for `/cmd_vel` to prevent command backlog during any residual network jitter.

Future Consideration: For deployment on a physical robot in the field, a wired Ethernet connection may not always be feasible. In such scenarios, a dedicated industrial-grade WiFi access point with strong

signal strength and minimal interference would be required. Alternatively, a point-to-point wireless bridge or 5G cellular link with low-latency guarantees could be explored. The current implementation's automatic reconnection logic and QoS policies provide a foundation for such wireless deployments.

6.2 Unity Graphics API Compatibility

Challenge: During development, the Unity editor and Meta XR Simulator repeatedly crashed when using DirectX 11 as the graphics API. The simulator would work for short periods (a few minutes) before freezing or crashing entirely. Multiple troubleshooting attempts including driver updates, graphics quality reductions, and project reconfiguration failed to resolve the instability.

Solution: The graphics API was switched from DirectX 11 to Vulkan. While Vulkan is not the default API for Meta Quest development, it proved stable on the development hardware. The simulator no longer crashed, enabling continuous testing and development. A minor trade-off was a slight reduction in visual quality compared to DirectX 11, but stability took priority over graphical fidelity for the teleoperation task.

Hardware Consideration: The instability is likely attributable to the development machine's GPU or driver limitations under the combined load of VR rendering, ROS2 message processing, and the simulator. On more capable hardware, DirectX 11 may perform without issues. The current Vulkan-based implementation provides a stable foundation that can be migrated to DirectX 11 or improved with better hardware in the future.

6.3 VR Controller Input Testing (Hardware Unavailability)

Challenge: Due to the unavailability of a physical Meta Quest headset, the primary VR controller input could not be tested. The `OVRInput.Get(OVRInput.Axis2D.PrimaryThumbstick)` method was implemented as intended for the final system, but the Meta XR Simulator's controller emulation did not reliably register thumbstick input on the development machine. Attempts to map the simulator's virtual controller to the expected OVRInput axes were unsuccessful.

Solution: A keyboard fallback was implemented to enable complete testing and demonstration of the teleoperation system. The fallback uses standard WASD keys: W moves forward, S moves backward, A turns left, and D turns right. This ensures that the entire teleoperation pipeline can be validated without a physical headset.

The primary VR controller code remains intact in the project. Based on the Meta XR SDK's API parity guarantee, the same input logic should function correctly on a physical Quest headset without modifications. However, this remains untested due to hardware constraints.

Demonstration Note: The system demonstration video shows teleoperation using the keyboard fallback. While the final intended use case is VR controller input, the keyboard fallback successfully validates all downstream components: command publication to `/cmd_vel`, Gazebo physics execution, odometry feedback to Unity, and VR visualization. The core teleoperation loop is fully functional; only the input device differs from the original specification.

7 Conclusion & Future Work

This project successfully implemented a cross-platform VR teleoperation system for mobile robots. The system integrates Unity (Windows) for VR visualization, ROS2 Jazzy and Gazebo (Ubuntu 24.04) for physics simulation, and RTAB-Map for offline environment mapping. A distributed architecture was established using the ROS-TCP-Connector and ROS-TCP-Endpoint bridge, enabling deterministic cross-platform communication.

The VR interface provides four switchable camera views (main, left wheel, right wheel, and following view), a 2D minimap with real-time robot position tracking, and color-coded distance-to-wall feedback using simulated LiDAR data. A dual-layer connection monitor tracks both TCP health and message freshness, while odometry-driven pose synchronization keeps the Unity robot perfectly aligned with Gazebo's ground-truth state.

All core features were validated through observable demonstration in the Meta XR Simulator. Although live mapping was part of the original project pitch, the static map approach proved more robust given computational and hardware constraints. The keyboard fallback successfully validates the complete teleoperation pipeline, and the primary VR controller code remains ready for deployment on physical Meta Quest hardware.

Several extensions could further improve the system for real-world deployment. Live SLAM integration would enable teleoperation in unmapped or dynamic environments without requiring a pre-built static mesh. Testing on physical hardware is necessary to assess wireless latency, operator comfort, and sensor noise in practical conditions. The robot-independent architecture naturally supports multi-robot teleoperation, allowing an operator to switch between multiple agents or coordinate their movements. Recording and replaying operator trajectories would also benefit autonomous navigation training and performance benchmarking.

For field deployment, replacing the current Ethernet connection with an industrial-grade WiFi access point or low-latency point-to-point wireless bridge would maintain communication reliability, leveraging the existing automatic reconnection logic. Upgrading to a more capable GPU would improve rendering stability and visual quality, potentially enabling DirectX 11 operation.

All code, configuration files, and exported assets are available in the project GitHub repository [6]. Overall, this work demonstrates a functional and extensible VR teleoperation framework that balances practical hardware constraints with immersive operator feedback, providing a solid foundation for future research and development in robot teleoperation.

References

[1] Meta. *Meta Quest Developer Guidelines: Performance Best Practices*. 2025. URL: https://developers.meta.com/horizon/documentation/native/pc/dg-performance-guidelines/.

[2] Open Robotics. *ROS 2 Jazzy Jalisco Documentation*. 2024. URL: https://docs.ros.org/en/ros2_documentation/jazzy.

[3] Unity Technologies. *ROS-TCP-Connector: Unity to ROS2 Bridge*. 2024. URL: https://github.com/Unity-Technologies/ROS-TCP-Connector.

[4] Unity Technologies. *ROS-TCP-Endpoint: ROS2 Node for Unity Communication*. 2024. URL: https://github.com/Unity-Technologies/ROS-TCP-Endpoint.

[5] Mathieu Labbé. *RTAB-Map: Exporting 3D Mesh for External Use*. 2024. URL: https://introlab.github.io/rtabmap.

[6] Rubin Khadka, Sarvenaz Ashoori, and Abdul Hayee Hafiz. *VR Controlled Robot Teleop*. 2026. URL: https://github.com/rubin-khadka/VR_Controlled_Robot_Teleop.

[7] Rubin Khadka, Sarvenaz Ashoori, and Abdul Hayee Hafiz. *VR Controlled Robot Teleoperation - Demonstration Video*. 2026. URL: https://github.com/rubin-khadka/VR_Controlled_Robot_Teleop#vr-teleoperation-demo--headset-perspective.